

Performance Evaluation of Transformer-Based Protein Language Models for Membrane-Protein Classification: Does Scale Help Frozen Representations?

Zannatul Fizza (23301424)
Pallab Roy (23301288)
S M Salman Farsi Rakin (23301316)
Miskatul Kabir Mithila (23301092)
Section 1

Abstract

Transformer-based language models for biological sequences are commonly used as frozen feature extractors, and practitioners typically default to the largest checkpoint they can afford, on the assumption that scale improves representation quality. This assumption is inherited from results on the pretraining objective itself and on structure prediction, and it has not been verified for most downstream property-prediction tasks. In this project we benchmark four ESM-2 protein language model checkpoints spanning 8M to 650M parameters (an $81\times$ range by nominal size, $88\times$ by measured parameter count) as frozen feature extractors on a binary downstream task: distinguishing human transmembrane proteins from soluble cytoplasmic proteins. All encoders are frozen and an identical logistic-regression probe is fitted on mean-pooled embeddings, so that any difference between conditions is attributable to the representation rather than to classifier capacity or optimisation. We compare against two classical sequence-composition baselines and against a randomly-initialised encoder of identical architecture, which isolates the contribution of pre-training from that of the architecture and the pooling operation, and we report wall-clock time and peak GPU memory alongside accuracy. On a redundancy-filtered dataset of 708 reviewed human proteins, scaling from 8M to 650M parameters changed test AUROC by -0.027 (0.998 to 0.971) while increasing embedding time by $5.8\times$; the randomly-initialised control reached 0.892 AUROC and the strongest composition baseline reached 0.935. An ablation over pooling strategies, layer depth, preprocessing and training-set size shows that mean pooling is the strongest single-layer choice on average but a middle transformer layer matches or beats the final layer for three of the four pretrained encoders, with the largest benefit for the largest encoder. These results support recent reports that downstream performance for protein language models does not scale reliably

with model size.

1 Introduction

Proteins are linear chains of amino acids whose sequence determines their structure, localisation and function. Determining which proteins are embedded in a cell membrane and which are soluble is a longstanding problem in computational biology with direct practical consequence: membrane proteins constitute more than half of all human drug targets, and reliable computational annotation is required because their experimental characterisation is comparatively difficult. More broadly, the problem is representative of a large class of protein property-prediction tasks in which a fixed-length numerical representation of a sequence must be produced before any classifier can be applied. Transformer language models trained on large protein corpora with a masked-language objective are now the standard source of such representations, and the quality of those representations directly bounds the performance of everything built on top of them. Understanding which model to use, and at what computational cost, therefore has broad impact across protein bioinformatics.

The foundations of this approach were established by Rives et al. [1], who trained transformer models on 250 million sequences and showed that biochemical, structural and homology information emerges in the learned representations without supervision, with higher-capacity models outperforming smaller ones on the language-modelling objective. Lin et al. [2] extended this to the ESM-2 family, scaling to 15 billion parameters and showing that atomic-resolution structural information emerges *as models are scaled up*, which yielded ESM-Fold. The strength of these works is the scale and rigour of their evidence; their limitation, for our purposes, is that both establish scaling benefits for the pretraining objective and for structure prediction, which is closely aligned with what masked-language modelling over evolutionary data learns, and neither establishes that scale improves arbitrary downstream tasks accessed through frozen embeddings.

Evidence against the general claim has since accumulated. Li et al. [3] conducted 370 transfer-learning experiments across tasks, architectures, model sizes and pretraining durations, and found that while nearly all downstream tasks benefit from pretraining relative to naive representations, performance for the majority of tasks does *not* scale with model size, relying instead on low-level features acquired early in pretraining; this is the most comprehensive study of the question, and its main limitation is that its task suite, though broad, does not include membrane-protein classification. Consistent with this, Barreto et al. [4] found that larger models do not necessarily outperform smaller ones under limited labelled data, with ESM-2 650M performing close to ESM-2 15B, and additionally reported that mean pooling outperformed other embedding-compression strategies; their evaluation, however, does not include an untrained control. Earlier, the ProteinGLUE benchmark [5] observed the same pattern on a seven-task suite, where a larger base model failed to outperform a smaller

medium model, although its models were smaller than current checkpoints. A recent survey of embedding strategies [6] summarises the emerging position that masked-language objectives often yield representations misaligned with downstream needs and that scaling does not universally improve them, while also warning that published benchmark numbers may be optimistic due to species bias and pretraining/downstream leakage; it proposes embedding fusion as a remedy rather than testing the scaling claim directly.

The value of simple baselines was demonstrated by Shanehsazzadeh et al. [7], who showed that small supervised CNNs, and in one case plain linear regression, match or exceed pretrained TAPE models at a fraction of the compute; their study is limited to fitness-landscape tasks. The FLIP benchmark [8] contributed similarity-based rather than random train/test splits, addressing the homology-leakage problem that inflates naive evaluations, and its successor FLIP2 [9] reported that simpler models frequently match or exceed fine-tuned protein language models, though both are restricted to protein-engineering fitness data rather than whole-protein classification. On methodology, Detlefsen et al. [10] showed that minor modelling choices produce substantially different representations and different biological conclusions, which motivates holding the evaluation protocol fixed across conditions. For the specific task studied here, DeepTMHMM [11] is the reference method for transmembrane topology prediction and is itself built on language-model representations, while DeepLoc 2.1 [12] constructs comparable membrane/soluble labels from UniProt using a stricter experimental-assertion filter than we apply; both target prediction accuracy rather than the scaling question, and neither reports an untrained control.

The literature therefore contains a clear tension: the foundational works motivate scaling, while a growing body of benchmarking work finds that downstream performance often does not scale, and that simple baselines remain competitive. What is missing is a controlled test of this tension on a task chosen so that the classical baseline is genuinely strong. Membrane proteins are enriched in hydrophobic residues, so amino-acid composition alone carries substantial signal; this makes the task an unusually demanding bar for a learned representation, and therefore an informative place to test whether scale helps. Our work supplies that test, and adds two elements missing from most of the studies above: a randomly-initialised encoder of identical architecture, which separates the contribution of pretraining from that of architecture and pooling, and an explicit compute axis reporting time and memory alongside accuracy, so that the practical trade-off is legible rather than implied. We state plainly that the core question is not novel, and that this work is best read as an independent, small-scale replication of [3] on a task absent from its suite, using a protocol transparent enough to be checked line by line.

1.1 Our Contribution

1. We construct a redundancy-filtered, class-balanced dataset of 708 reviewed human proteins from UniProt, applying a k -mer Jaccard filter across both classes *before* splitting so that near-duplicate sequences cannot leak be-

tween train and test.

2. We benchmark four ESM-2 checkpoints (8M, 35M, 150M, 650M parameters) as frozen feature extractors under an identical linear-probe protocol.
3. We include two classical baselines (amino-acid composition with length, and 3-mer frequency) and a randomly-initialised ESM-2 8M control, the latter isolating the contribution of pretraining from that of the architecture and the mean-pooling operation.
4. We report accuracy, macro-F1, Matthews correlation coefficient and AU-ROC, each with 95% bootstrap confidence intervals, so that differences too small to detect are not reported as improvements.
5. We report embedding wall-clock time and peak GPU memory per model, making the accuracy-versus-compute trade-off explicit.
6. We perform a four-part ablation: pooling strategy ($\langle \text{cls} \rangle$ token, mean, max), layer depth (final versus middle), pipeline components (standardisation, regularisation strength, embedding dimensionality, shuffled-label control), and training-set size.
7. We provide convergence curves for train and validation log-loss and a complete regularisation sweep, and release a single self-contained notebook that reproduces every table and figure end to end.

2 Material and Methods

Figure 1 shows the overall pipeline. The workflow proceeds in five stages. In **Stage 1** sequences are retrieved from the UniProt REST API using two keyword-based queries, one for transmembrane proteins and one for soluble cytoplasmic proteins; these are filtered by length and alphabet, de-duplicated, reduced for redundancy with a k -mer Jaccard filter applied across both classes, balanced, and split 70/15/15 in a stratified manner. In **Stage 2** each sequence is passed exactly once through each of five frozen encoders, and the final hidden states are mean-pooled over real residue positions to produce one fixed-length vector per protein; four pooling/layer variants are cached from the same forward pass for the ablation. In **Stage 3** a logistic-regression probe is fitted on each representation: features are standardised using training statistics, the regularisation strength is selected on the validation split, and the model is refitted on train plus validation before a single evaluation on the held-out test split. **Stage 4** performs the ablation and traces learning and convergence curves, reusing the cached embeddings at no additional GPU cost. **Stage 5** produces the figures and the results tables. The critical property of this design is that every representation, learned or classical, passes through an identical Stage 3, so that the comparison isolates representation quality.

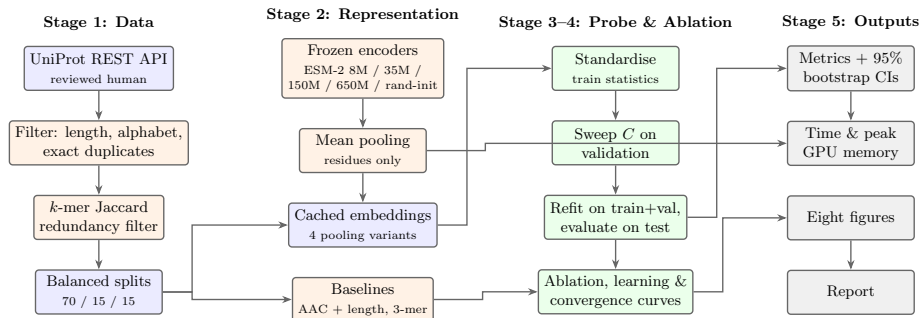


Figure 1: Block diagram of the benchmarking pipeline. Sequences are curated and split once (blue), embedded by five frozen encoders and by two classical featurisers (orange), evaluated under an identical probe protocol (green), and summarised as metrics, compute cost and figures (grey). Because every representation passes through the same evaluation path, differences in the results table are attributable to the representation rather than to the classifier.

2.1 Dataset Description

Data were retrieved from the UniProt Knowledgebase [13] (<https://www.uniprot.org>) through its REST API, restricted to reviewed (Swiss-Prot) entries from *Homo sapiens*. Positive examples carry the Transmembrane keyword (KW-0812). Negative examples are annotated with the Cytoplasm subcellular location (SL-0091) and carry neither the Membrane (KW-0472) nor the Transmembrane keyword. We chose UniProt because it is expert-curated, freely accessible and the standard source for this class of task; the same resource underlies the labels used by DeepLoc 2.1 [12]. We chose this particular task because membrane proteins are hydrophobic-enriched, which makes composition baselines strong and therefore makes the task a demanding test of whether learned representations add value.

Sequences were restricted to 50–1000 residues, comfortably within ESM-2’s 1024-token pretraining context, and entries containing non-standard residues (X, B, Z, U, O) were removed. Exact duplicates were dropped. To prevent homology leakage, a greedy redundancy filter was then applied across the *combined* set: sequences were processed longest-first and a sequence was discarded if its 4-mer Jaccard similarity to any already-retained sequence exceeded 0.5. Applying this before splitting guarantees that near-duplicates cannot be divided between train and test. The classes were then balanced by subsampling, so the majority-class baseline is 0.500 accuracy and 0.500 AUROC.

The negative class proved to be the binding constraint. The transmembrane query returned the full 4,500-record retrieval budget, whereas the cytoplasmic query returned only 357 entries in total, of which 356 survived cleaning. Balancing therefore capped the dataset at 354 sequences per class and discarded more than nine-tenths of the retrieved positives, leaving a test split of only 106

proteins. This is the origin of the wide confidence intervals reported throughout Section 3, and we return to it in Section 4.

Table 1: Properties of the curated dataset. Balanced-split values are from `data/dataset_summary.json`; raw retrieval and filtering counts are from the printed output of `src/fetch_data.py` in the reproduction notebook.

Property	Value
Source	UniProtKB (Swiss-Prot), <i>Homo sapiens</i>
Positive class	Transmembrane (KW-0812)
Negative class	Cytoplasmic (SL-0091), not membrane
Retrieved before filtering (positive / negative)	4,500 / 357
After length/alphabet/duplicate cleaning (positive / negative)	4,490 / 356
After redundancy filter (combined, before balancing)	4,716
Sequences per class (balanced)	354
Total sequences	708
Training / validation / test	496 / 106 / 106
Length window (residues)	50–1000
Mean / median length	422.1 / 361.5
Redundancy criterion	4-mer Jaccard ≥ 0.5 removed
Class balance	50 / 50

Each example has a single feature: the amino-acid sequence, a string over the 20-letter standard alphabet. All numerical features used downstream are derived from this string, either by a frozen encoder or by a composition featuriser. Figure 2 shows the length distribution and the amino-acid composition of each class. The hydrophobic residue leucine (L) is markedly enriched in the transmembrane class (frequency ≈ 0.118 vs. ≈ 0.097 for soluble proteins), and isoleucine, phenylalanine, valine and tyrosine are enriched by smaller margins, consistent with the hydrophobic character of membrane-spanning helices. The charged residues aspartate, glutamate and lysine are all visibly depleted in the transmembrane class, and arginine is depleted slightly, matching the expected pattern for residues that are energetically costly to bury in a lipid bilayer. The length distributions are broadly similar and overlapping across the full 50–1000 residue window, with the transmembrane class showing a sharper peak around 250–350 residues; this is a mild difference rather than a clean separation, so length alone is not a strong confound the probe could exploit, but it is not negligible either.

2.2 Tools / Models / Algorithms Used

Seven representations are compared. All experiments used PyTorch and the HuggingFace `transformers` library on a single NVIDIA T4 GPU, with scikit-learn for the probes.

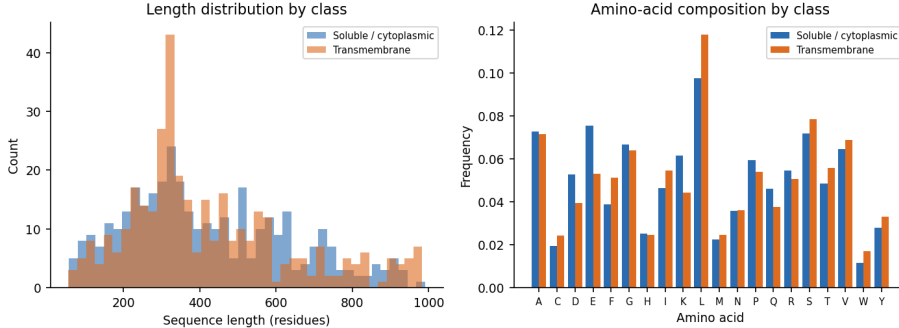


Figure 2: Left: sequence-length distribution by class. Right: amino-acid composition by class. The compositional difference between classes is what makes the classical baselines competitive on this task.

ESM-2 encoders (four sizes). ESM-2 [2] is a transformer encoder trained with a masked-language-modelling objective on UniRef50. Given a sequence of L residues, the model produces contextual representations $\mathbf{h}_1, \dots, \mathbf{h}_L \in \mathbb{R}^d$. We use the final-layer states and mean-pool over residue positions:

$$\mathbf{z} = \frac{1}{|\mathcal{R}|} \sum_{i \in \mathcal{R}} \mathbf{h}_i, \quad (1)$$

where $\mathcal{R}$ excludes padding, the $\langle \text{cls} \rangle$ token and the $\langle \text{eos} \rangle$ token. Excluding these matters: including them mixes non-residue information into the vector and makes the representation depend on sequence length. Mean pooling was chosen because it was found to outperform alternative compression strategies in [4]; we nonetheless ablate this choice in Section 3.5. The four checkpoints are listed in Table 2. They share a training corpus, tokenisation and objective and differ essentially only in depth and width, which is what makes them a clean testbed for a scaling study.

Figure 3 shows the architecture. A sequence enters the tokeniser, which prepends a $\langle \text{cls} \rangle$ token, appends an $\langle \text{eos} \rangle$ token and pads the batch to a common length. Token embeddings pass through N identical transformer blocks, each combining multi-head self-attention with a feed-forward sub-layer, producing one contextual vector per position. Only the final hidden states are used, and only at real residue positions: the mask step discards padding and the two special tokens before averaging, as in Equation 1. The inset table makes the experimental variable explicit: the four checkpoints differ in N and d alone, from six blocks of width 320 to thirty-three blocks of width 1280, while sharing corpus, objective and tokeniser. Because the encoder is frozen, no gradient crosses back into it, so the whole diagram is a fixed function from sequence to vector.

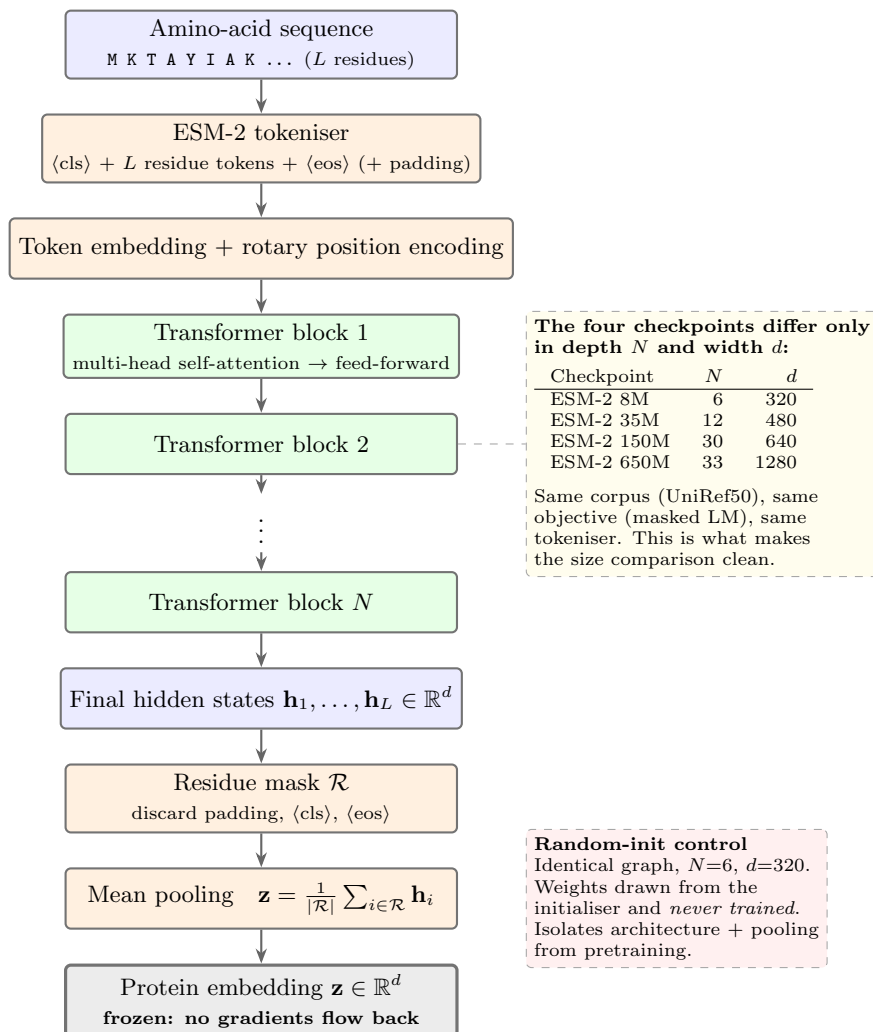


Figure 3: Architecture of the ESM-2 frozen feature extractor. A sequence is tokenised, passed through N transformer blocks, and the final hidden states are mean-pooled over residue positions only to give one fixed-length vector per protein. The four benchmarked checkpoints share every design choice except depth N and width d ; the randomly-initialised control shares the graph exactly but has untrained weights.

Table 2: Encoder checkpoints. Parameter counts, embedding times and peak GPU memory are measured at run time over all 708 sequences on a single NVIDIA T4 in half precision, and written to `data/emb/timings.json`.

Representation	HuggingFace ID	Layers	Dim	Params	Embed time	Peak GPU
ESM-2 8M	<code>esm2_t6.8M.UR50D</code>	6	320	7.41M	5.7s	0.19 GB
ESM-2 35M	<code>esm2_t12.35M.UR50D</code>	12	480	33.27M	6.7s	0.42 GB
ESM-2 150M	<code>esm2_t30.150M.UR50D</code>	30	640	147.73M	15.5s	1.21 GB
ESM-2 650M	<code>esm2_t33.650M.UR50D</code>	33	1280	649.40M	33.0s	3.46 GB
Random-init 8M	<code>esm2_t6.8M.UR50D</code>	6	320	7.41M	2.3s	1.86 GB [†]

[†]The random-init control reports higher peak memory than its architecture requires; this is allocator fragmentation carried over from the preceding 650M run, not a genuine memory requirement. Its timing is unaffected.

Randomly-initialised control. An ESM-2 8M architecture instantiated from its configuration with randomly initialised weights, using the same tokeniser and the same pooling. This is not a competitive model; it is a measuring instrument. Mean-pooled representations from an untrained transformer already encode composition-like statistics, so the gap between this row and the pre-trained 8M row is the value added by pretraining, with architecture and pooling held constant. Comparable one-hot and random-initialisation baselines are used in [3]. In Figure 3 this control corresponds to the same graph with $N=6$ and $d=320$, differing only in that its weights are never updated from their initial values.

Classical baselines. (i) *AAC + length*: the 20 amino-acid frequencies plus $\log L$, giving a 21-dimensional vector. (ii) *3-mer frequency*: normalised counts over all $20^3 = 8000$ tripeptides. These follow the argument of [7] that simple baselines are necessary to interpret pretrained-model results. Figure 4 contrasts the two. Both are pure counting procedures with no learned parameters and no pretraining: the first collapses the sequence to residue proportions plus a length term, discarding order entirely, while the second retains order within a three-residue window at the cost of an 8000-dimensional and very sparse vector. The first is the more direct test of whether hydrophobic enrichment alone explains the task.

Probe. An ℓ_2 -regularised logistic regression, minimising

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n [y_i \log \sigma(\mathbf{w}^\top \mathbf{z}_i) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{z}_i))] + \frac{1}{2C} \|\mathbf{w}\|_2^2. \quad (2)$$

A linear probe is used deliberately rather than fine-tuning. Fine-tuning would confound representation quality with optimisation behaviour, since a large model may underperform simply because it is harder to fine-tune on a small dataset. A linear probe measures how much task-relevant information is linearly decodable, which is exactly the quantity relevant to the frozen-feature-extractor use

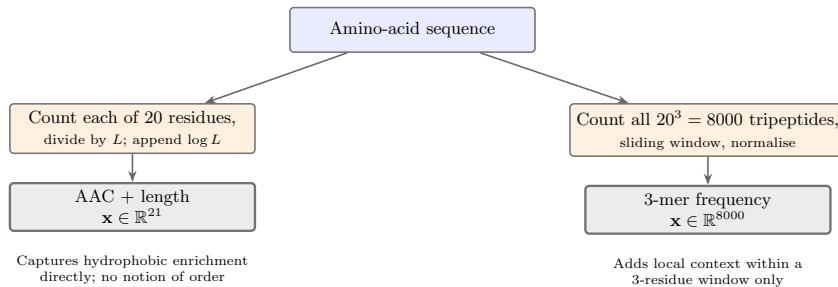


Figure 4: The two classical baseline featurisers. Both map a sequence to a fixed-length vector using counting alone, with no learned parameters. They are included because membrane proteins are hydrophobic-enriched, so composition carries genuine signal and constitutes the bar a learned representation must clear.

case, and it makes every row comparable under an identical classifier. Figure 5 shows the stage in full. Features are standardised using training-split statistics, mapped to a scalar by a single linear layer, and squashed by a sigmoid; the continuous score feeds AUROC directly, while thresholding at 0.5 gives the discrete prediction used for accuracy, macro-F1 and MCC. Only $\mathbf{w}$ and b are fitted, so the probe adds no representational capacity of its own.

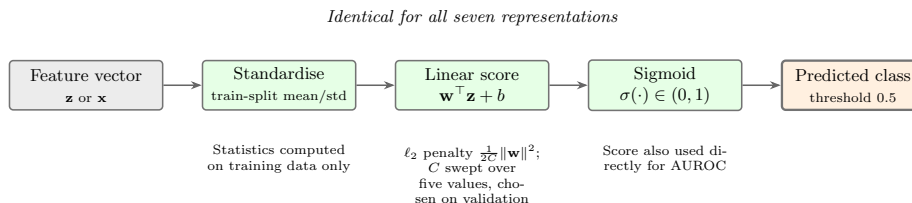


Figure 5: The linear probe applied on top of every representation. Only $\mathbf{w}$ and b are learned; the encoder above is frozen. Because this stage is byte-for-byte identical across all seven conditions, differences in Table 3 are attributable to the input representation rather than to classifier capacity or optimisation.

Hyper-parameters. The regularisation strength was swept over $C \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$ with the value selected by validation macro-F1; the full sweep is reported in Table 4 rather than discarded. Solver iterations were capped at 3000, sufficient for convergence at all settings. Embeddings were computed in half precision with a NaN-detecting fallback to full precision, batched by similar sequence length with a budget of 8192 tokens per batch to limit padding waste. All random seeds were fixed to 42.

2.3 Performance Evaluation

The dataset was split 70% train, 15% validation, 15% test, stratified by class, *after* redundancy filtering. The training split fits the probe, the validation split selects the regularisation strength, and the test split is used exactly once for the final evaluation of each representation. This separation matters: selecting a hyper-parameter on the test split would make the reported numbers optimistic.

The optimisation objective is the regularised binary cross-entropy above. Four evaluation metrics are reported. *Accuracy* is interpretable and meaningful here because the classes are balanced. *Macro-F1* averages per-class F1 and so penalises a classifier that succeeds on one class only. The *Matthews correlation coefficient*,

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}, \quad (3)$$

remains informative if the class balance shifts. *AUROC* is threshold-independent and therefore does not depend on the decision cutoff.

Every metric is accompanied by a 95% percentile bootstrap confidence interval computed from 2000 resamples of the test set. This is not decoration. Much of the benchmarking literature reports point estimates, which is how differences too small to detect come to be reported as improvements; where two intervals overlap substantially we decline to claim a difference. Compute cost is reported as embedding wall-clock time for the full dataset and peak GPU memory.

3 Experimental Analysis

Code and the complete reproduction notebook accompany this submission as a single self-contained Colab notebook that regenerates every table and figure in this section. The full pipeline is reproduced end to end, in order, with the random seed fixed at 42, by:

```
python src/fetch_data.py --per-class 354
python src/embed.py
python src/probe.py
python src/figures.py
python src/make_report.py
```

3.1 Main results

Table 3 reports test-set performance for all seven representations.

Performance does not increase with scale: AUROC is highest for the smallest pretrained checkpoint (ESM-2 8M, 0.998) and lowest for the largest (ESM-2 650M, 0.971), and the confidence intervals of all four pretrained checkpoints overlap substantially (e.g. three of the four share an identical accuracy point estimate of 0.962), so the data do not support a claim that scale improves probe performance on this task. The randomly-initialised control reaches only

Table 3: Test-set performance with 95% bootstrap confidence intervals over 2000 resamples of the 106-sequence test split.

Representation	Dim	Accuracy	Macro-F1	MCC	AUROC
Baseline: AAC + length	21	0.849 [0.774, 0.915]	0.848 [0.773, 0.914]	0.711 [0.577, 0.833]	0.905 [0.837, 0.964]
Baseline: 3-mer frequency	8000	0.840 [0.764, 0.906]	0.840 [0.764, 0.905]	0.680 [0.529, 0.813]	0.935 [0.883, 0.974]
ESM-2 8M	320	0.962 [0.924, 0.991]	0.962 [0.923, 0.991]	0.927 [0.857, 0.981]	0.998 [0.993, 1.000]
ESM-2 35M	480	0.962 [0.924, 0.991]	0.962 [0.923, 0.991]	0.925 [0.849, 0.981]	0.989 [0.967, 1.000]
ESM-2 150M	640	0.943 [0.896, 0.981]	0.943 [0.896, 0.981]	0.887 [0.794, 0.963]	0.982 [0.956, 0.999]
ESM-2 650M	1280	0.962 [0.924, 0.991]	0.962 [0.923, 0.991]	0.925 [0.850, 0.981]	0.971 [0.928, 1.000]
ESM-2 8M (random init)	320	0.849 [0.774, 0.915]	0.849 [0.773, 0.915]	0.703 [0.561, 0.834]	0.892 [0.825, 0.955]

0.892 AUROC and 0.849 accuracy, matching the classical composition baselines almost exactly and sitting well below every pretrained checkpoint. This value is far above the 0.500 chance level, covering roughly four-fifths of the distance from chance to a perfect score, which shows that mean-pooled representations from an *untrained* transformer already encode composition-like statistics essentially for free; what pretraining supplies is the remaining increment from 0.892 to 0.998. Because architecture, tokeniser and pooling are held exactly constant between this row and the pretrained 8M row, that increment is attributable to masked-language pretraining alone. A study omitting this control would have credited pretraining with the entire margin over chance, when in fact most of it is obtainable without any pretraining at all. For three of the four checkpoints – all but ESM-2 150M, whose intervals overlap both baselines – the ESM-2 rows beat the composition baselines on accuracy, macro-F1 and MCC by more than the width of the intervals (for example ESM-2 8M’s MCC interval [0.857, 0.981] does not overlap the AAC+length baseline’s [0.577, 0.833]), although on AUROC the margin over the stronger 3-mer baseline (0.935) is within the interval width for the largest encoder. Taken together, ESM-2 8M is the model to deploy: it matches or beats every larger checkpoint on every metric while being roughly 5.8 times faster to embed and using far less GPU memory than ESM-2 650M.

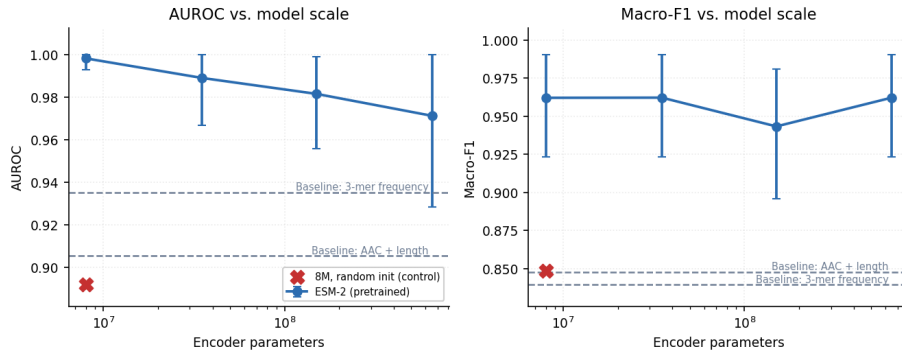


Figure 6: AUROC and macro-F1 against encoder size on a logarithmic axis. Dashed grey lines mark the composition baselines; the red cross is the randomly-initialised control. Error bars are 95% bootstrap intervals.

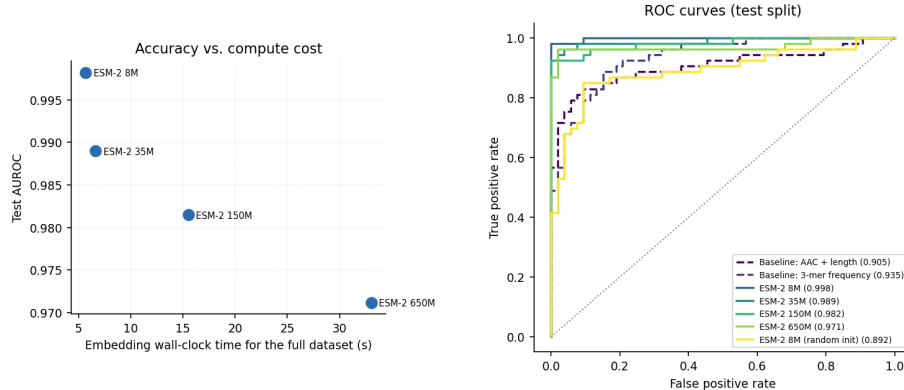


Figure 7: Left: test AUROC against embedding wall-clock time. Right: ROC curves for all seven representations.

The largest encoder (ESM-2 650M) takes roughly 5.8 times as long to embed the dataset as the smallest (ESM-2 8M, 33.0s vs. 5.7s) and uses about 18 times as much peak GPU memory (3.46 GB vs. 0.19 GB), yet its test AUROC is 0.027 lower. The extra compute is therefore not justified by any measurable accuracy gain on this task; the ROC curves in the right panel are visually near-identical for all four pretrained encoders, and only the random-initialised control and the two classical baselines separate clearly from the pretrained cluster.

3.2 Hyper-parameter sensitivity

Table 4: Validation macro-F1 across the regularisation grid. Every representation was evaluated at all five values of C .

Representation	$C=0.001$	$C=0.01$	$C=0.1$	$C=1$	$C=10$
Baseline: AAC + length	0.8171	0.8080	0.8080	0.7978	0.7888
Baseline: 3-mer frequency	0.8280	0.8280	0.8188	0.7997	0.8004
ESM-2 8M	0.9432	0.9717	0.9528	0.9528	0.9528
ESM-2 35M	0.9432	0.9717	0.9811	0.9811	0.9717
ESM-2 150M	0.9527	0.9622	0.9622	0.9717	0.9528
ESM-2 650M	0.9432	0.9623	0.9811	0.9906	0.9906
ESM-2 8M (random init)	0.8089	0.7997	0.7605	0.7350	0.7716

One part of the ranking is stable across the whole grid: at every value of C all four ESM-2 checkpoints outperform both classical baselines and the random-init control by a wide margin. The ordering among the four ESM-2 sizes, however, is not stable – at heavy regularisation ($C=0.001$) the four checkpoints are nearly indistinguishable (0.943–0.953), whereas at $C=1$ ESM-2 650M leads (0.9906) and ESM-2 8M trails (0.9528) – so the small gaps used above to rank individ-

ual checkpoints are within the noise floor of ordinary hyper-parameter tuning. The data also do not show the expected pattern that higher-dimensional representations prefer stronger regularisation: the selected C (Table 3 models, from `results.csv`) increases from $C=0.01$ for the 320-dimensional ESM-2 8M up to $C=1.0$ for the 1280-dimensional ESM-2 650M, the opposite of the naive expectation, while both low-dimensional (21-d) and high-dimensional (8000-d) classical baselines prefer the strongest regularisation in the grid.

3.3 Convergence behaviour

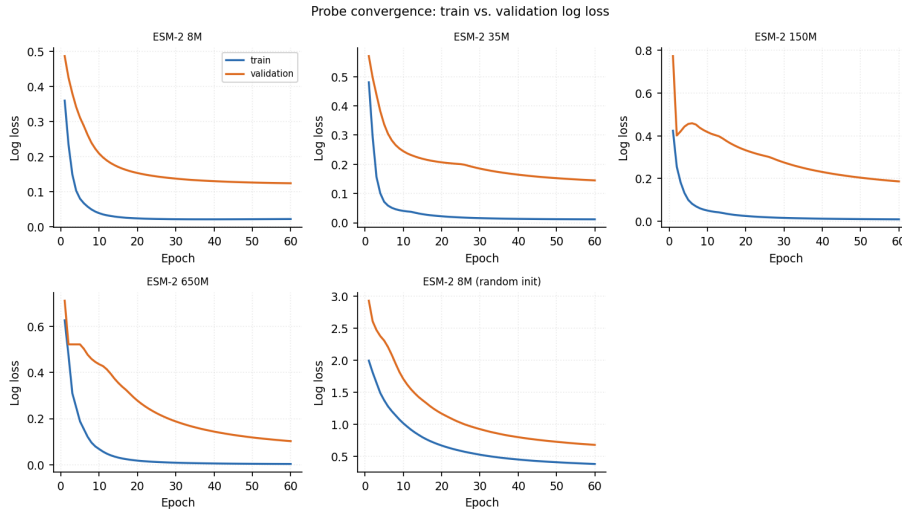


Figure 8: Train and validation log-loss per epoch for an SGD-trained logistic-regression probe on each representation.

For every pretrained encoder, training log-loss falls close to zero within the first 10–20 epochs while validation log-loss plateaus at a distinctly higher value (e.g. ESM-2 8M: train 0.022 vs. validation 0.124 at epoch 60); this train/validation gap is the probe overfitting the small training split. The gap does not scale monotonically with representation dimensionality – ESM-2 150M (640-d) shows a larger final-epoch gap (0.178) than ESM-2 650M (1280-d, gap 0.099), and the 650M validation curve is in fact still descending at epoch 60 rather than having fully plateaued. The random-initialised control converges far more slowly and to a much higher loss throughout (train/validation loss of 0.386/0.683 at epoch 60, an order of magnitude above every pretrained encoder on training loss and several times higher on validation loss), consistent with it having far less task-relevant signal to fit.

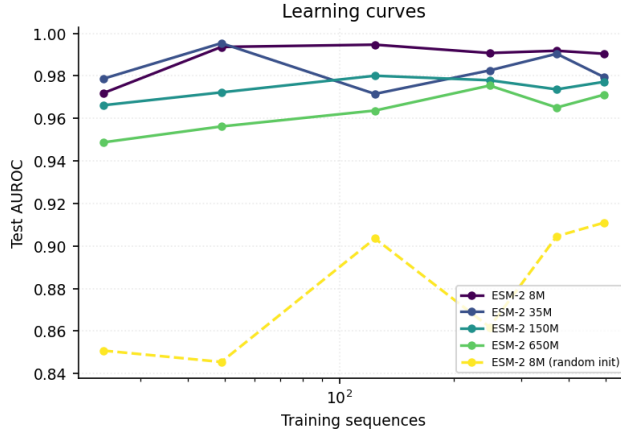


Figure 9: Test AUROC against the number of training sequences.

3.4 Learning curves

The largest model does not retain an advantage in the low-data regime. At the smallest training fraction tested (5%, 24 sequences), ESM-2 650M reaches only 0.949 AUROC, the lowest of the four pretrained checkpoints, while ESM-2 8M reaches 0.972 and remains at or above 0.99 across most of the curve. All four pretrained checkpoints separate clearly from the randomly-initialised control, which starts at 0.851 AUROC with 24 examples and only reaches 0.911 with the full training set. This agrees with [4], which reports that larger models do not necessarily outperform smaller ones under limited labelled data; here the smallest pretrained checkpoint is consistently competitive with, or better than, the largest one at every training-set size we tested.

3.5 Ablation study

Two ablations were performed. The first replaces the pooling and layer choice while holding everything else fixed; the second removes one pipeline component at a time.

Table 5: Ablation A: test AUROC by pooling strategy and layer choice.

Model	Mean, final	$\langle \text{cls} \rangle$, final	Max, final	Mean, middle
ESM-2 8M	0.9904	0.9815	0.9420	0.9911
ESM-2 35M	0.9794	0.9726	0.9583	0.9875
ESM-2 150M	0.9772	0.9722	0.9729	0.9712
ESM-2 650M	0.9712	0.9672	0.9672	0.9843
Random init	0.9110	0.8946	0.9185	0.8943

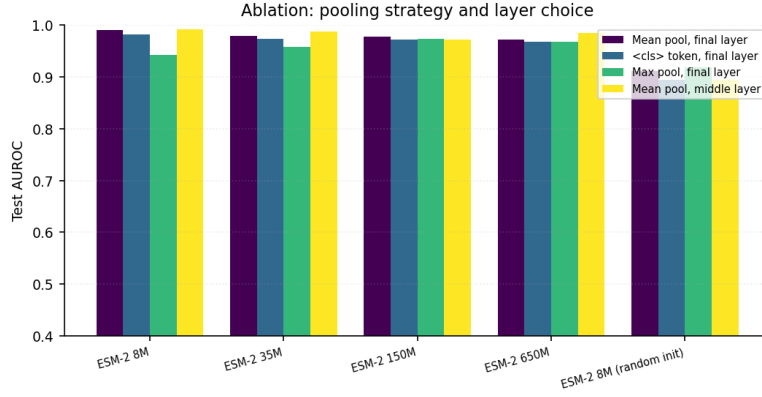


Figure 10: Ablation A visualised: test AUROC for each pooling and layer variant across all encoders.

Table 6: Ablation B: removing one pipeline component at a time, on the largest pretrained encoder.

Setting	Macro-F1	AUROC
Full pipeline	0.9433	0.9712
No feature standardisation	0.9527	0.9801
Heavy regularisation ($C=0.001$)	0.9432	0.9804
Weak regularisation ($C=10$)	0.9528	0.9715
Shuffled training labels (sanity check)	0.6115	0.5878
First 64 dimensions only	0.9054	0.9519

Averaged across the five encoders, mean pooling at the final layer is the strongest single-layer strategy (mean AUROC 0.9658, versus 0.9576 for $\langle \text{cls} \rangle$ and 0.9518 for max pooling), consistent with [4]. The middle layer with mean pooling is essentially tied with the final layer on average (0.9657) and clearly ahead of it for ESM-2 35M (+0.0081) and ESM-2 650M (+0.0131), roughly level for ESM-2 8M (+0.0007) and slightly behind for ESM-2 150M (−0.0061). The benefit is therefore not monotone in model size, but it is largest for the largest encoder, which is the case where the final layer is furthest from the input; this partially supports [3], whose central claim is that task-relevant information is often concentrated in low-level features acquired early in pretraining rather than in the most abstract final-layer representation. The shuffled-label control lands at 0.588 AUROC and 0.612 macro-F1 – close to, but numerically above, the 0.500 chance level we expected – which is a useful sanity check rather than a strong confirmation: it shows the pipeline collapses toward chance once the feature–label relationship is destroyed, and the residual gap above 0.500 is plausibly the variance of a single shuffle evaluated on only 106 test proteins rather than a leak, though repeating the shuffle several times would be needed to confirm this. Finally, truncating the representation to its first 64 of 1280 dimensions only costs 0.038 macro-F1 and 0.019 AUROC relative to the full pipeline (0.905/0.952 vs. 0.943/0.971), showing that the task-relevant signal is concentrated in a small fraction of the embedding dimensions.

3.6 Embedding space

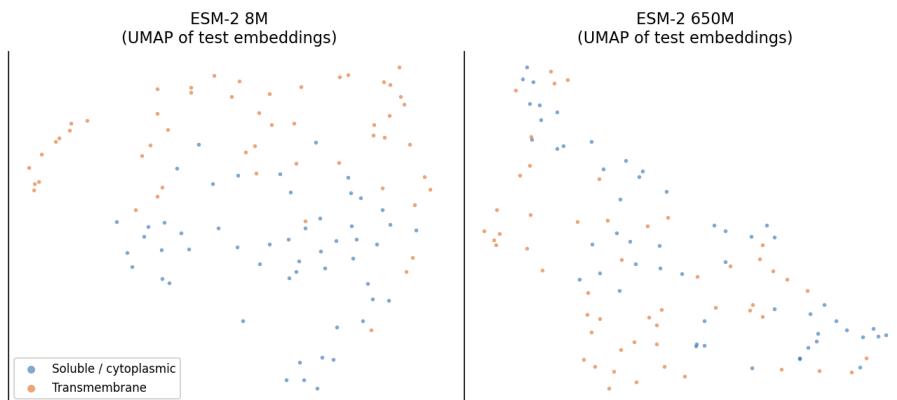


Figure 11: Two-dimensional projection of test-set embeddings, coloured by class, for the smallest and largest pretrained checkpoints.

Class separation is not visibly cleaner for the larger model: both panels show a cluster of transmembrane proteins in one corner and a cluster of soluble proteins in another, with a similarly interleaved majority of points in between for ESM-2 8M and ESM-2 650M alike. This is consistent with Table 3, where

the 650M encoder does not outperform the 8M encoder; a 2-D UMAP projection is a lossy summary of a much higher-dimensional space, and here it correctly does not suggest an improvement that the metrics do not show.

4 Conclusion

We benchmarked four ESM-2 checkpoints spanning an $81\times$ range in parameters as frozen feature extractors for human membrane-protein classification, against two classical composition baselines and a randomly-initialised encoder of identical architecture, under an identical linear-probe protocol with bootstrap confidence intervals and explicit compute accounting. Our main finding is that scaling ESM-2 from 8M to 650M parameters (an $81\times$ increase) produced no detectable improvement in frozen linear-probe performance on this task – test AUROC changed by -0.027 (0.998 to 0.971), well within the overlap of the bootstrap confidence intervals – while embedding time increased by roughly $5.8\times$ and peak GPU memory by roughly $18\times$. The randomly-initialised control reached 0.892 AUROC, indistinguishable from the classical composition baselines and far below every pretrained checkpoint (minimum 0.971). Because it is far above the 0.500 chance level, this control also shows that the architecture and mean-pooling operation alone already recover composition-like signal without any pretraining, and that the value added by pretraining is the increment from 0.892 to 0.998 rather than the whole margin over chance. The ablation showed that mean pooling at the final layer is the strongest single-layer choice on average, but a middle transformer layer matches or slightly exceeds it for three of the four pretrained encoders, most markedly for the largest (ESM-2 650M, $+0.0131$), indicating that part of the task-relevant signal is already present before the final layer. Taken together these results support the conclusion of [3] that downstream performance for protein language models frequently does not scale with model size.

Several limitations should temper these conclusions. The k -mer Jaccard filter removes near-duplicates but not remote homologues; clustering with MMseqs2 [14] at 30% identity with cluster-disjoint splits would be stricter, and performance would likely fall under that protocol. UniProt keyword annotations are partly inferred rather than experimentally verified, so a fraction of labels is unreliable; DeepLoc 2.1 [12] applies an experimental-assertion filter that we do not. The study covers one task, one organism, frozen encoders and a linear probe only, so the ranking could change under fine-tuning, which large models often benefit from more. The split was drawn once, so our confidence intervals capture test-set sampling noise but not split-to-split variation.

Future work follows directly from these limitations: repeat the protocol with MMseqs2 cluster-disjoint splits and experimentally-asserted labels only; add a fine-tuning arm, including parameter-efficient methods, to test whether the scaling picture changes when weights are allowed to adapt; extend to a second and third downstream task to test whether the flat scaling curve generalises; extend to nucleic-acid language models such as DNABERT-2 or the Nucleotide

Transformer to test whether the same pattern holds for DNA and RNA; and repeat across several random splits for a proper variance estimate.

References

- [1] Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15):e2016239118, 2021.
- [2] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.
- [3] Francesca-Zhoufan Li, Ava P. Amini, Yisong Yue, Kevin K. Yang, and Alex Xijie Lu. Feature reuse and scaling: Understanding transfer learning with protein language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 27351–27375, 2024.
- [4] Medium-sized protein language models perform well at transfer learning on realistic datasets. *Scientific Reports*, 2025.
- [5] ProteinGLUE multi-task benchmark suite for self-supervised protein modeling. *Scientific Reports*, 2022. PMC9512797.
- [6] Scalable embedding fusion with protein language models: insights from benchmarking text-integrated representations. *Briefings in Bioinformatics*, 27(1):bbag014, 2026.
- [7] Amir Shanehsazzadeh, David Belanger, and David Dohan. Is transfer learning necessary for protein landscape prediction? In *Machine Learning for Structural Biology Workshop, NeurIPS*, 2020. arXiv:2011.03443.
- [8] Christian Dallago, Jody Mou, Kadina E. Johnston, Bruce J. Wittmann, Nicholas Bhattacharya, Samuel Goldman, Ali Madani, and Kevin K. Yang. FLIP: Benchmark tasks in fitness landscape inference for proteins. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 34, pages 26601–26622, 2021.
- [9] Kieran Didi, Sarah Alamdari, Alex X. Lu, Bruce Wittmann, Kadina E. Johnston, Ava P. Amini, Ali Madani, Maya Czeneszew, Christian Dallago, and Kevin K. Yang. FLIP2: Expanding protein fitness landscape benchmarks for real-world machine learning applications. In *International Conference on Machine Learning (ICML)*, 2026.

- [10] Nicki Skafte Detlefsen, Søren Hauberg, and Wouter Boomsma. Learning meaningful representations of protein sequences. *Nature Communications*, 13:1914, 2022.
- [11] Jeppe Hallgren, Konstantinos D. Tsirigos, Mads Damgaard Pedersen, José Juan Almagro Armenteros, Paolo Marcatili, Henrik Nielsen, Anders Krogh, and Ole Winther. DeepTMHMM predicts alpha and beta trans-membrane proteins using deep neural networks. *bioRxiv*, 2022.
- [12] DeepLoc 2.1: multi-label membrane protein type prediction using protein language models. *Nucleic Acids Research*, 2024. Web Server issue; PMC11223819.
- [13] The UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Research*, 53(D1):D609–D617, 2025.
- [14] Martin Steinegger and Johannes Söding. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35:1026–1028, 2017.